

Công cụ - matplotlib

Notebook này hướng dẫn cách sử dụng thư viện matplotlib để vẽ các biểu đồ đẹp mắt.



Table of Contents

- [1 Plotting your first graph](#)
- [2 Line style and color](#)
- [3 Saving a figure](#)
- [4 Subplots](#)
- [5 Multiple figures](#)
- [6 Pyplot's state machine: implicit vs explicit](#)
- [7 Pylab vs Pyplot vs Matplotlib](#)
- [8 Drawing text](#)
- [9 Legends](#)
- [10 Non-linear scales](#)
- [11 Ticks and tickers](#)
- [12 Polar projection](#)
- [13 3D projection](#)
- [14 Scatter plot](#)
- [15 Lines](#)
- [16 Histograms](#)
- [17 Images](#)
- [18 Animations](#)
- [19 Saving animations to video files](#)
- [20 What's next?](#)

✓ Vẽ biểu đồ đầu tiên của bạn

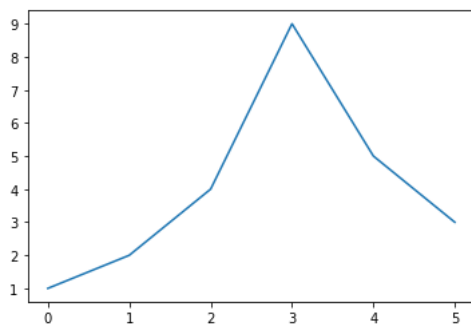
Trước tiên, chúng ta cần import thư viện `matplotlib`.

```
import matplotlib
```

Bây giờ hãy cùng vẽ biểu đồ đầu tiên! :)

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 4, 9, 5, 3])
plt.show()
```



Đúng vậy, việc này đơn giản như việc gọi hàm `plot` với dữ liệu, và sau đó gọi hàm `show`!

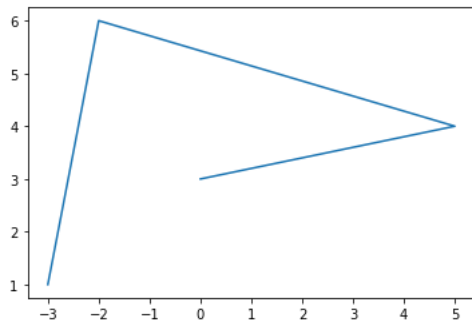
Ghi chú:

Matplotlib có thể xuất biểu đồ bằng nhiều thư viện đồ họa backend khác nhau như Tk, wxPython, v.v. Khi chạy Python bằng dòng lệnh, bạn có thể muốn chỉ định backend nào sẽ được sử dụng ngay sau khi import matplotlib và trước khi vẽ bất kỳ thứ

gi. Ví dụ: để sử dụng backend Tk, hãy chạy `matplotlib.use("TKAgg")`. Tuy nhiên, trong Jupyter notebook, mọi thứ dễ dàng hơn: việc import `import matplotlib.pyplot` sẽ tự động đăng ký chính Jupyter làm backend, vì vậy các biểu đồ sẽ hiển thị trực tiếp trong notebook. Trước đây việc này yêu cầu chạy `%matplotlib inline`, nên bạn vẫn có thể thấy lệnh đó trong một số notebook, nhưng hiện tại nó không còn cần thiết nữa.

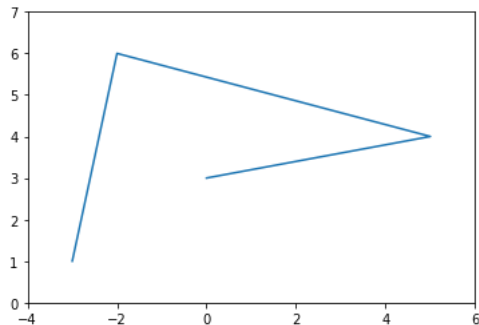
Nếu hàm `plot` được cung cấp một mảng dữ liệu, nó sẽ sử dụng mảng đó làm tọa độ trên trục tung (vertical axis), và sử dụng chỉ số (index) của mỗi điểm dữ liệu trong mảng làm tọa độ ngang (horizontal coordinate). Bạn cũng có thể cung cấp hai mảng: một cho trục ngang (`x`), và mảng thứ hai cho trục dọc (`y`):

```
plt.plot([-3, -2, 5, 0], [1, 6, 4, 3])
plt.show()
```



Các trục sẽ tự động khớp với phạm vi của dữ liệu. Chúng ta muốn để lại một chút khoảng trống cho biểu đồ, vì vậy hãy gọi hàm `axis` để thay đổi phạm vi của mỗi trục `[xmin, xmax, ymin, ymax]`.

```
plt.plot([-3, -2, 5, 0], [1, 6, 4, 3])
plt.axis([-4, 6, 0, 7])
plt.show()
```

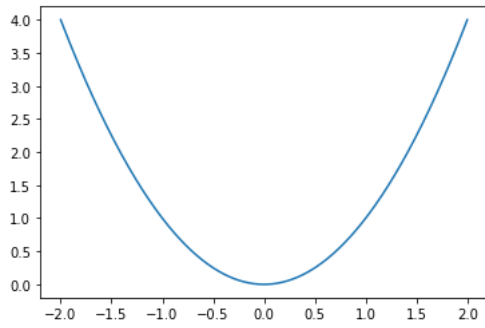


Bây giờ, hãy thử vẽ một hàm toán học. Chúng ta sử dụng hàm `linspace` của NumPy để tạo một mảng `x` chứa 500 số thực ngẫu nhiên trong khoảng từ -2 đến 2, sau đó tạo mảng thứ hai `y` được tính bằng bình phương của `x`.

```
import numpy as np

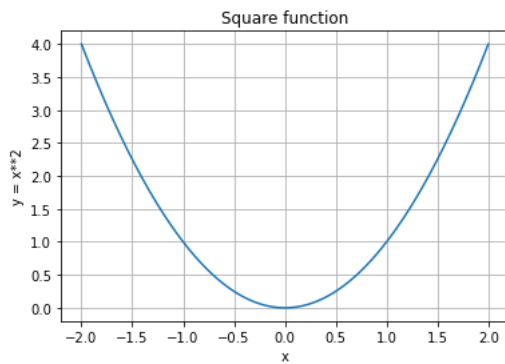
x = np.linspace(-2, 2, 500)
y = x**2

plt.plot(x, y)
plt.show()
```



Biểu đồ hơi đơn điệu, hãy thêm tiêu đề, nhãn cho trục x, trục y và vẽ thêm lưới (grid).

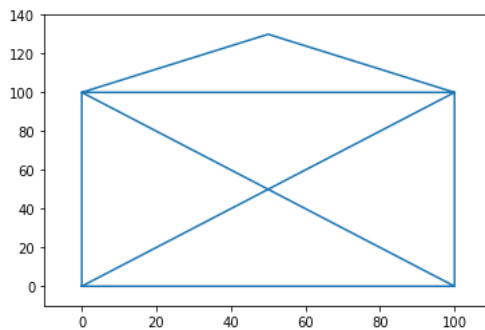
```
plt.plot(x, y)
plt.title("Square function")
plt.xlabel("x")
plt.ylabel("y = x**2")
plt.grid(True)
plt.show()
```



✓ Kiểu đường và màu sắc

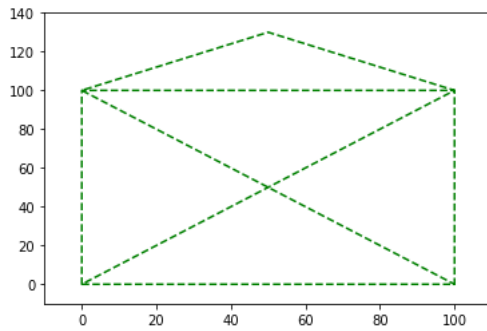
Theo mặc định, matplotlib vẽ một đường thẳng nối giữa các điểm liên tiếp.

```
plt.plot([0, 100, 100, 0, 0, 100, 50, 0, 100],
         [0, 0, 100, 100, 0, 100, 130, 100, 0])
plt.axis([-10, 110, -10, 140])
plt.show()
```



Bạn có thể truyền đối số thứ 3 để thay đổi kiểu và màu sắc của đường. Ví dụ "g--" nghĩa là "đường nét đứt màu xanh lá cây" (green dashed line).

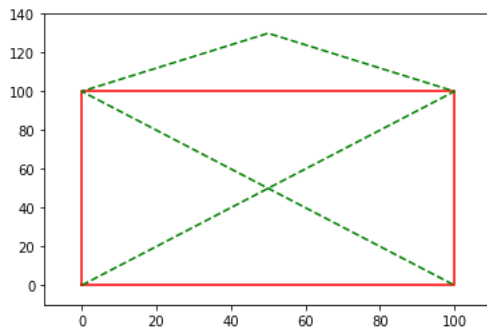
```
plt.plot([0, 100, 100, 0, 0, 100, 50, 0, 100],
         [0, 0, 100, 100, 0, 100, 130, 100, 0],
         "g--")
plt.axis([-10, 110, -10, 140])
plt.show()
```



Bạn có thể vẽ nhiều đường trên cùng một biểu đồ rất đơn giản: chỉ cần truyền `x1, y1, [style1], x2, y2, [style2], ...`

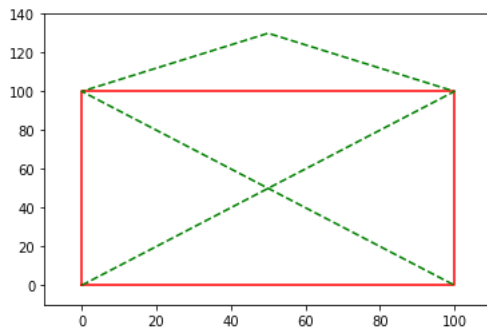
Ví dụ:

```
plt.plot([0, 100, 100, 0, 0], [0, 0, 100, 100, 0], "r-",
         [0, 100, 50, 0, 100], [0, 100, 130, 100, 0], "g--")
plt.axis([-10, 110, -10, 140])
plt.show()
```



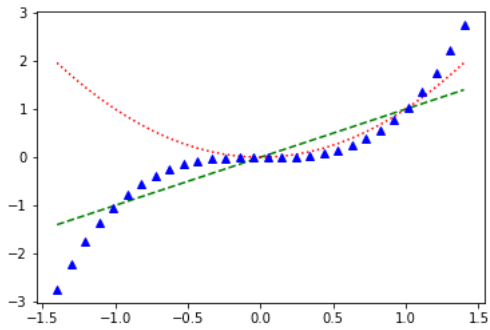
Or simply call `plot` multiple times before calling `show`.

```
plt.plot([0, 100, 100, 0, 0], [0, 0, 100, 100, 0], "r-")
plt.plot([0, 100, 50, 0, 100], [0, 100, 130, 100, 0], "g--")
plt.axis([-10, 110, -10, 140])
plt.show()
```



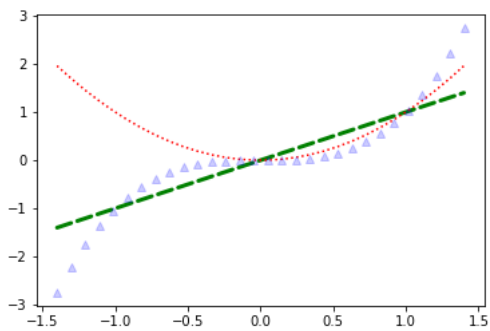
Bạn cũng có thể vẽ các điểm đơn giản thay vì các đường thẳng. Dưới đây là ví dụ với nét đứt màu xanh lá cây, đường chấm màu đỏ và các hình tam giác màu xanh dương. Hãy kiểm tra [tài liệu hướng dẫn](#) để xem danh sách đầy đủ các tùy chọn kiểu dáng & màu sắc.

```
x = np.linspace(-1.4, 1.4, 30)
plt.plot(x, x, 'g--', x, x**2, 'r:', x, x**3, 'b^')
plt.show()
```



Hàm plot trả về một danh sách các đối tượng `Line2D` (mỗi đối tượng cho một đường). Bạn có thể thiết lập thêm các thuộc tính cho các đường này, chẳng hạn như độ dày của đường (line width), kiểu nét đứt (dash style) hoặc mức độ trong suốt (alpha level).

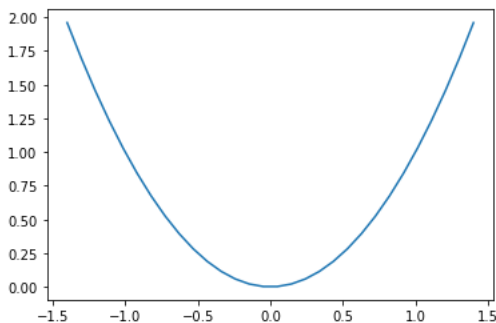
```
x = np.linspace(-1.4, 1.4, 30)
line1, line2, line3 = plt.plot(x, x, 'g--', x, x**2, 'r:', x, x**3, 'b^')
line1.set_linewidth(3.0)
line1.set_dash_capstyle("round")
line3.set_alpha(0.2)
plt.show()
```



✓ Lưu biểu đồ

Việc lưu biểu đồ vào đĩa đơn giản là gọi hàm `savefig` với tên tệp. Các định dạng hình ảnh có sẵn tùy thuộc vào backend đồ họa bạn sử dụng.

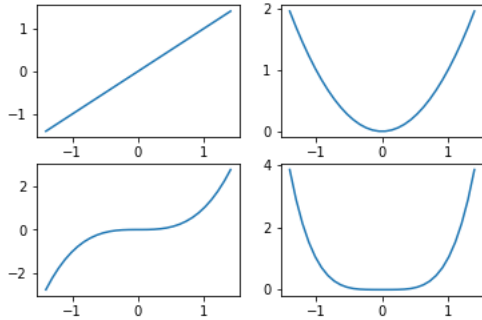
```
x = np.linspace(-1.4, 1.4, 30)
plt.plot(x, x**2)
plt.savefig("my_square_function.png", transparent=True)
```



✓ Biểu đồ con (Subplots)

Một figure trong matplotlib có thể chứa nhiều biểu đồ con (subplots). Các subplot này được tổ chức thành một lưới. Để tạo một subplot, chỉ cần gọi hàm `subplot`, và chỉ định số hàng, số cột trong figure, và chỉ số (index) của subplot bạn muốn vẽ (bắt đầu từ 1, sau đó từ trái sang phải, và từ trên xuống dưới). Lưu ý rằng pyplot theo dõi subplot hiện đang hoạt động, vì vậy khi bạn gọi hàm `plot`, nó sẽ vẽ trên subplot đang hoạt động đó.

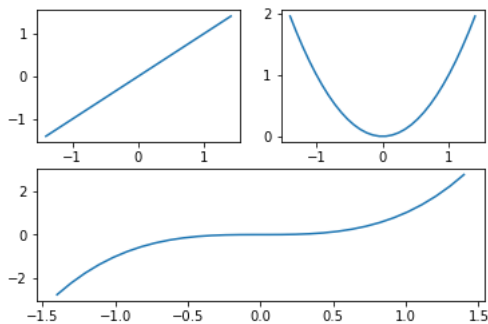
```
x = np.linspace(-1.4, 1.4, 30)
plt.subplot(2, 2, 1) # 2 rows, 2 columns, 1st subplot = top left
plt.plot(x, x)
plt.subplot(2, 2, 2) # 2 rows, 2 columns, 2nd subplot = top right
plt.plot(x, x**2)
plt.subplot(2, 2, 3) # 2 rows, 2 columns, 3rd subplot = bottom left
plt.plot(x, x**3)
plt.subplot(2, 2, 4) # 2 rows, 2 columns, 4th subplot = bottom right
plt.plot(x, x**4)
plt.show()
```



- Note that `subplot(223)` is a shorthand for `subplot(2, 2, 3)`.

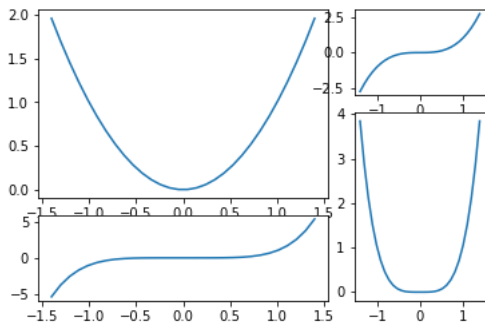
It is easy to create subplots that span across multiple grid cells like so:

```
plt.subplot(2, 2, 1) # 2 rows, 2 columns, 1st subplot = top left
plt.plot(x, x)
plt.subplot(2, 2, 2) # 2 rows, 2 columns, 2nd subplot = top right
plt.plot(x, x**2)
plt.subplot(2, 1, 2) # 2 rows, *1* column, 2nd subplot = bottom
plt.plot(x, x**3)
plt.show()
```



If you need more complex subplot positioning, you can use `subplot2grid` instead of `subplot`. You specify the number of rows and columns in the grid, then your subplot's position in that grid (top-left = (0,0)), and optionally how many rows and/or columns it spans. For example:

```
plt.subplot2grid((3,3), (0, 0), rowspan=2, colspan=2)
plt.plot(x, x**2)
plt.subplot2grid((3,3), (0, 2))
plt.plot(x, x**3)
plt.subplot2grid((3,3), (1, 2), rowspan=2)
plt.plot(x, x**4)
plt.subplot2grid((3,3), (2, 0), colspan=2)
plt.plot(x, x**5)
plt.show()
```



If you need even more flexibility in subplot positioning, check out the [corresponding matplotlib tutorial](#).

✓ Nhiều Figure

Cũng có thể vẽ trên nhiều figure khác nhau. Mỗi figure có thể chứa một hoặc nhiều subplot. Theo mặc định, matplotlib tự động tạo `figure(1)`. Khi bạn chuyển đổi figure, pyplot sẽ theo dõi figure hiện đang hoạt động, và subplot đang hoạt động của figure đó trở thành subplot hiện tại.

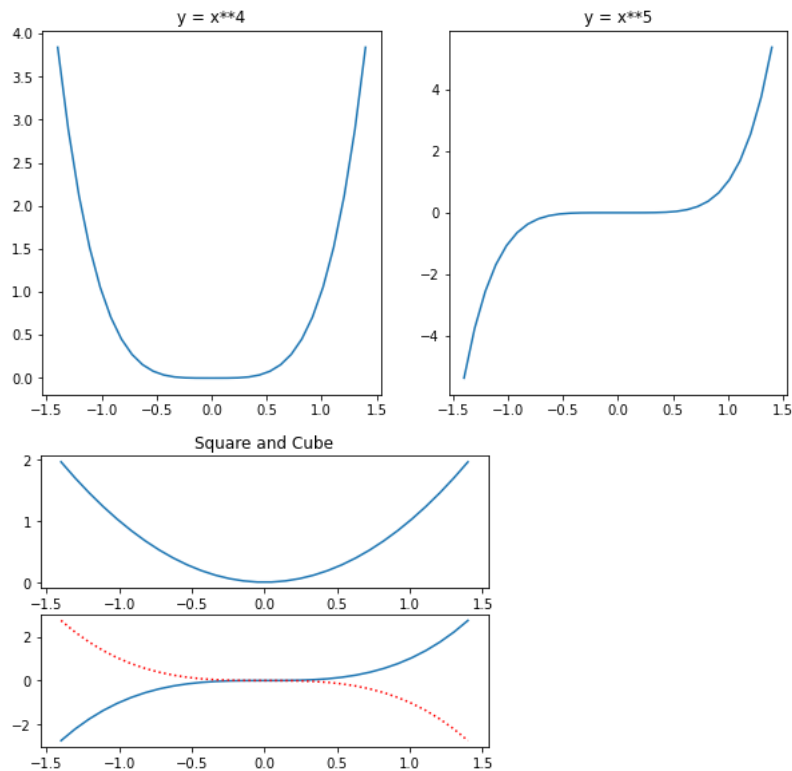
```
x = np.linspace(-1.4, 1.4, 30)

plt.figure(1)
plt.subplot(211)
plt.plot(x, x**2)
plt.title("Square and Cube")
plt.subplot(212)
plt.plot(x, x**3)

plt.figure(2, figsize=(10, 5))
plt.subplot(121)
plt.plot(x, x**4)
plt.title("y = x**4")
plt.subplot(122)
plt.plot(x, x**5)
plt.title("y = x**5")

plt.figure(1)      # back to figure 1, current subplot is 212 (bottom)
plt.plot(x, -x**3, "r:")

plt.show()
```



▼ State machine của Pyplot: ngầm định (implicit) đối vs tường minh (explicit)

Cho đến nay chúng ta đã sử dụng state machine của Pyplot để theo dõi các subplot hiện đang hoạt động. Mỗi khi bạn gọi hàm `plot`, pyplot chỉ vẽ lên subplot hiện tại. Nó cũng thực hiện một số bước tự động, như tạo figure và subplot khi bạn gọi `plot` nếu chúng chưa tồn tại. Sự tự động này rất tiện lợi trong môi trường tương tác (như Jupyter).

Nhưng khi bạn viết một chương trình, *tường minh tốt hơn là ngầm định*. Mã nguồn tường minh thường dễ gỡ lỗi và bảo trì hơn.

import this

The Zen of Python, by Tim Peters

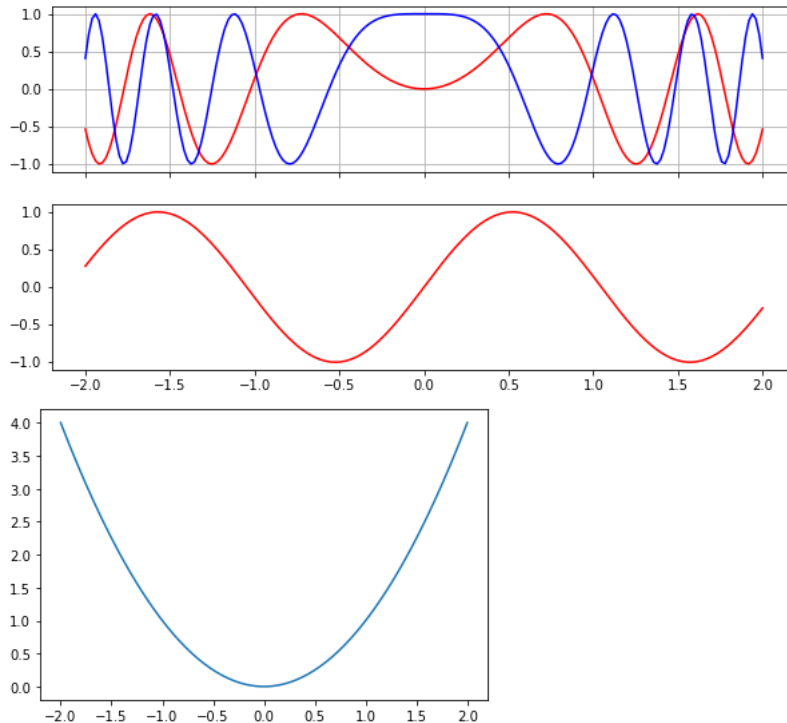
Beautiful is better than ugly.
 Explicit is better than implicit.
 Simple is better than complex.
 Complex is better than complicated.
 Flat is better than nested.
 Sparse is better than dense.
 Readability counts.
 Special cases aren't special enough to break the rules.
 Although practicality beats purity.
 Errors should never pass silently.
 Unless explicitly silenced.
 In the face of ambiguity, refuse the temptation to guess.
 There should be one-- and preferably only one --obvious way to do it.
 Although that way may not be obvious at first unless you're Dutch.
 Now is better than never.
 Although never is often better than *right* now.
 If the implementation is hard to explain, it's a bad idea.
 If the implementation is easy to explain, it may be a good idea.
 Namespaces are one honking great idea -- let's do more of those!

May mắn thay, Pyplot cho phép bạn bỏ qua hoàn toàn state machine, để bạn có thể viết mã nguồn tường minh đẹp mắt. Chỉ cần gọi hàm `subplots` và sử dụng đối tượng figure cùng danh sách các đối tượng axes được trả về. Ví dụ:

```
x = np.linspace(-2, 2, 200)
fig1, (ax_top, ax_bottom) = plt.subplots(2, 1, sharex=True)
fig1.set_size_inches(10,5)
line1, line2 = ax_top.plot(x, np.sin(3*x**2), "r-", x, np.cos(5*x**2), "b-")
line3 = ax_bottom.plot(x, np.sin(3*x), "r-")
ax_top.grid(True)
```



```
fig2, ax = plt.subplots(1, 1)
ax.plot(x, x**2)
plt.show()
```



For consistency, we will continue to use pyplot's state machine in the rest of this tutorial, but we recommend using the object-oriented interface in your programs.

PyLab vs Pyplot vs Matplotlib

There is some confusion around the relationship between pylab, pyplot and matplotlib. It's simple: matplotlib is the full library, it contains everything including pylab and pyplot.

Pyplot provides a number of tools to plot graphs, including the state-machine interface to the underlying object-oriented plotting library.

Pylab is a convenience module that imports matplotlib.pyplot and NumPy within a single namespace. You will find many examples using pylab, but it is now [strongly discouraged](#) (because *explicit* imports are better than *implicit* ones).

✓ Vẽ văn bản

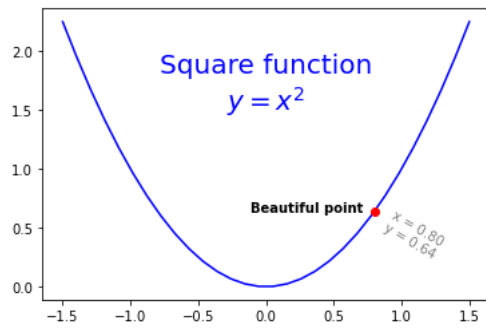
Bạn có thể gọi hàm `text` để thêm văn bản tại bất kỳ vị trí nào trong biểu đồ. Chỉ cần chỉ định tọa độ ngang, dọc và nội dung văn bản. Bất kỳ văn bản nào trong matplotlib cũng có thể chứa các biểu thức phương trình TeX.

```
x = np.linspace(-1.5, 1.5, 30)
px = 0.8
py = px**2

plt.plot(x, x**2, "b-", px, py, "ro")

plt.text(0, 1.5, "Square function\n$y = x^2$", fontsize=20, color='blue',
         horizontalalignment="center")
plt.text(px - 0.08, py, "Beautiful point", ha="right", weight="heavy")
plt.text(px + 0.05, py - 0.4, "x = %0.2f\ny = %0.2f"%(px, py), rotation=-30,
         color='gray')

plt.show()
```

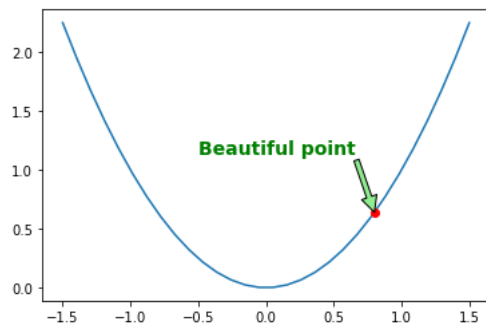


- Note: `ha` is an alias for `horizontalalignment`

For more text properties, visit [the documentation](#).

Every so often it is required to annotate elements of a graph, such as the beautiful point above. The `annotate` function makes it easy: just indicate the location of the point of interest, and the position of the text, plus optionally some extra arguments for the text and the arrow.

```
plt.plot(x, x**2, px, py, "ro")
plt.annotate("Beautiful point", xy=(px, py), xytext=(px-1.3, py+0.5),
            color="green", weight="heavy", fontsize=14,
            arrowprops={"facecolor": "lightgreen"})
plt.show()
```



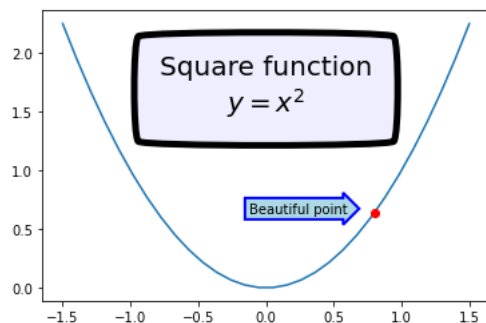
You can also add a bounding box around your text by using the `bbox` argument:

```
plt.plot(x, x**2, px, py, "ro")

bbox_props = dict(boxstyle="arrow,pad=0.3", ec="b", lw=2, fc="lightblue")
plt.text(px-0.2, py, "Beautiful point", bbox=bbox_props, ha="right")

bbox_props = dict(boxstyle="round4,pad=1,rounding_size=0.2", ec="black",
                    fc="#EEEEFF", lw=5)
plt.text(0, 1.5, "Square function\n$y = x^2$", fontsize=20, color='black',
        ha="center", bbox=bbox_props)

plt.show()
```



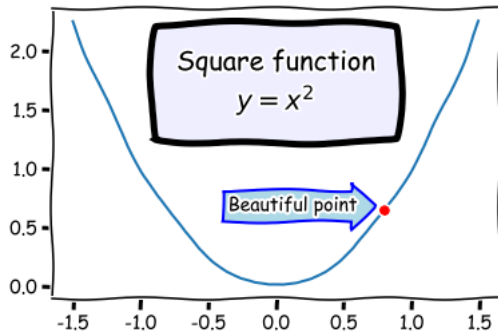
Just for fun, if you want an [xkcd](#)-style plot, just draw within a `with plt.xkcd()` section:

```
with plt.xkcd():
    plt.plot(x, x**2, px, py, "ro")

    bbox_props = dict(boxstyle="arrow,pad=0.3", ec="b", lw=2, fc="lightblue")
    plt.text(px-0.2, py, "Beautiful point", bbox=bbox_props, ha="right")

    bbox_props = dict(boxstyle="round4,pad=1,rounding_size=0.2", ec="black",
                          fc="#EEEEFF", lw=5)
    plt.text(0, 1.5, "Square function\n$y = x^2$", fontsize=20, color='black',
             ha="center", bbox=bbox_props)

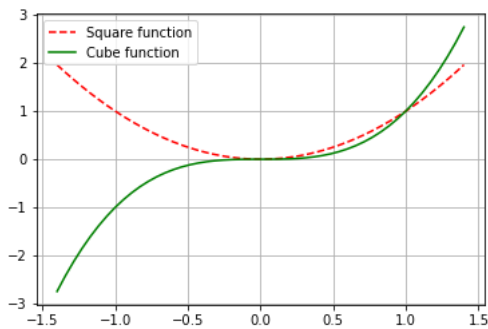
plt.show()
```



✓ Chú giải (Legends)

Cách đơn giản nhất để thêm chú giải là đặt nhãn (`label`) cho tất cả các đường, sau đó chỉ cần gọi hàm `legend`.

```
x = np.linspace(-1.4, 1.4, 50)
plt.plot(x, x**2, "r--", label="Square function")
plt.plot(x, x**3, "g-", label="Cube function")
plt.legend(loc="best")
plt.grid(True)
plt.show()
```



✓ Thang đo phi tuyến tính

Matplotlib hỗ trợ các thang đo phi tuyến tính (non-linear scales), chẳng hạn như thang đo logarit hoặc logit.

```
x = np.linspace(0.1, 15, 500)
y = x**3/np.exp(2*x)

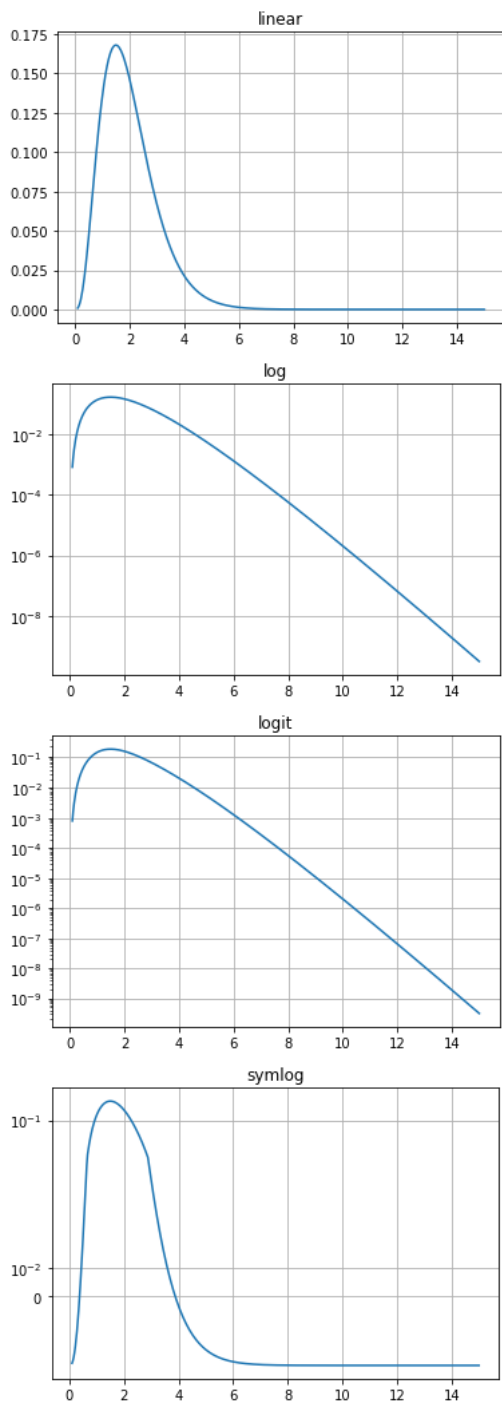
plt.figure(1)
plt.plot(x, y)
plt.yscale('linear')
plt.title('linear')
plt.grid(True)

plt.figure(2)
plt.plot(x, y)
plt.yscale('log')
plt.title('log')
plt.grid(True)
```

```
plt.figure(3)
plt.plot(x, y)
plt.yscale('logit')
plt.title('logit')
plt.grid(True)

plt.figure(4)
plt.plot(x, y - y.mean())
plt.yscale('symlog', linthresh=0.05)
plt.title('symlog')
plt.grid(True)

plt.show()
```



✓ Ticks và tickers

Các trục có các dấu nhỏ gọi là "ticks". Chính xác thì, "ticks" là *vị trí* của các dấu (vd: -1, 0, 1), "tick lines" là các vạch nhỏ được vẽ tại các vị trí đó, "tick labels" là các nhãn được vẽ cạnh tick lines, và "tickers" là các đối tượng có khả năng quyết định vị trí đặt ticks.

Đôi khi bạn cần kiểm soát nhiều hơn (vd: có quá nhiều nhãn tick trên đồ thị logit ở trên). Matplotlib cung cấp cho bạn toàn quyền kiểm soát các ticks. Bạn thậm chí có thể kích hoạt các minor ticks (vạch phụ).

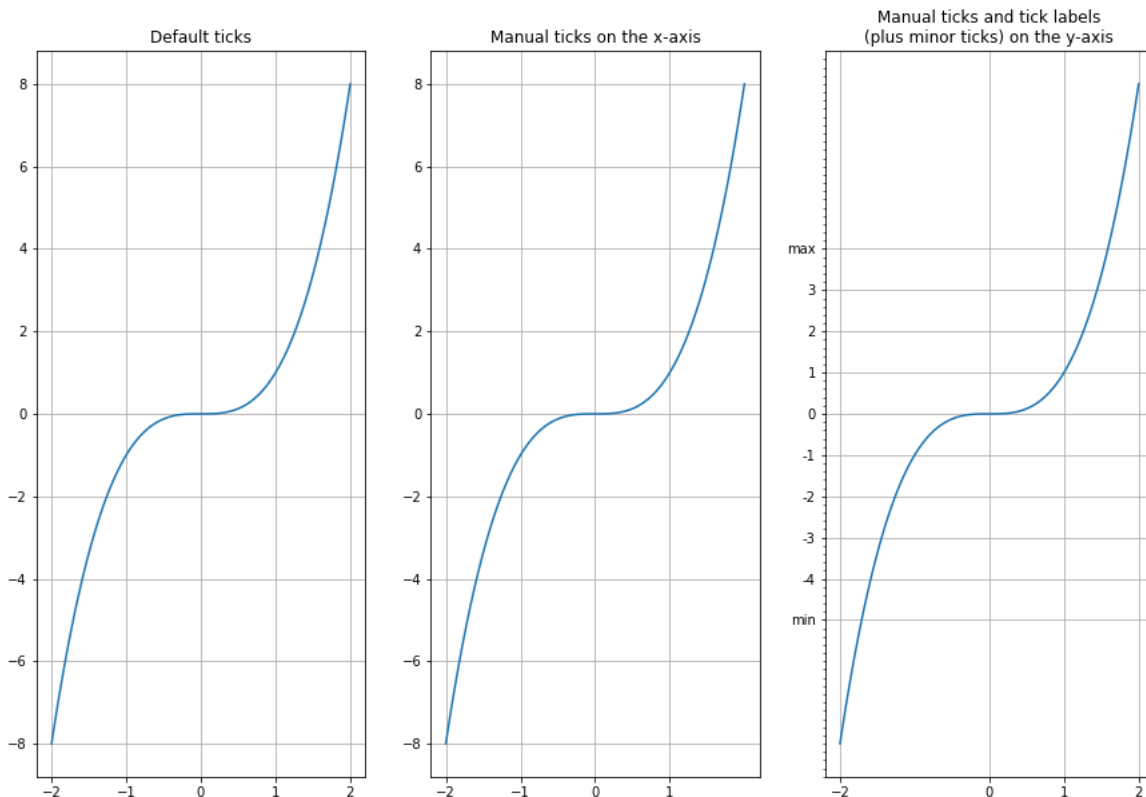
```
x = np.linspace(-2, 2, 100)

plt.figure(1, figsize=(15,10))
plt.subplot(131)
plt.plot(x, x**3)
plt.grid(True)
plt.title("Default ticks")

ax = plt.subplot(132)
plt.plot(x, x**3)
ax.xaxis.set_ticks(np.arange(-2, 2, 1))
plt.grid(True)
plt.title("Manual ticks on the x-axis")

ax = plt.subplot(133)
plt.plot(x, x**3)
plt.minorticks_on()
ax.tick_params(axis='x', which='minor', bottom=False)
ax.xaxis.set_ticks([-2, 0, 1, 2])
ax.yaxis.set_ticks(np.arange(-5, 5, 1))
ax.yaxis.set_ticklabels(["min", -4, -3, -2, -1, 0, 1, 2, 3, "max"])
plt.grid(True)
plt.title("Manual ticks and tick labels\n(plus minor ticks) on the y-axis")

plt.show()
```



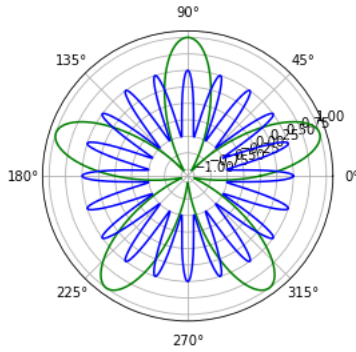
✓ Polar projection

Drawing a polar graph is as easy as setting the `projection` argument to `"polar"` when creating the subplot.

```
radius = 1
theta = np.linspace(0, 2*np.pi*radius, 1000)

plt.subplot(111, projection='polar')
```

```
plt.plot(theta, np.sin(5*theta), "g-")
plt.plot(theta, 0.5*np.cos(20*theta), "b-")
plt.show()
```

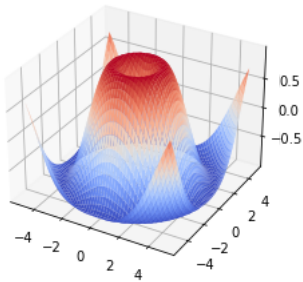


3D projection

Plotting 3D graphs is quite straightforward: when creating a subplot, set the `projection` to `"3d"`. It returns a 3D axes object, which you can use to call `plot_surface`, providing x, y, and z coordinates, plus other optional arguments. For more information on generating 3D plots, check out the [matplotlib tutorial](#).

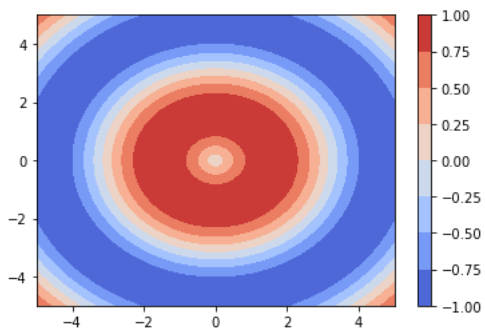
```
x = np.linspace(-5, 5, 50)
y = np.linspace(-5, 5, 50)
X, Y = np.meshgrid(x, y)
R = np.sqrt(X**2 + Y**2)
Z = np.sin(R)

figure = plt.figure(1, figsize = (12, 4))
subplot3d = plt.subplot(111, projection='3d')
surface = subplot3d.plot_surface(X, Y, Z, rstride=1, cstride=1,
                                cmap=matplotlib.cm.coolwarm, linewidth=0.1)
plt.show()
```



Another way to display this same data is *via* a contour plot.

```
plt.contourf(X, Y, Z, cmap=matplotlib.cm.coolwarm)
plt.colorbar()
plt.show()
```

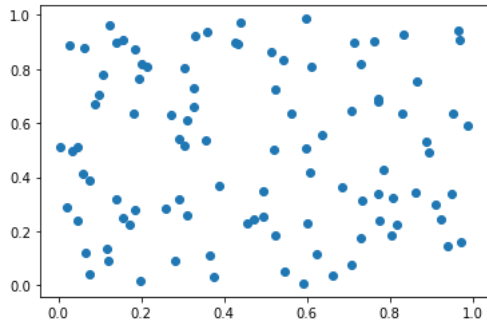


✓ Scatter plot

To draw a scatter plot, simply provide the x and y coordinates of the points.

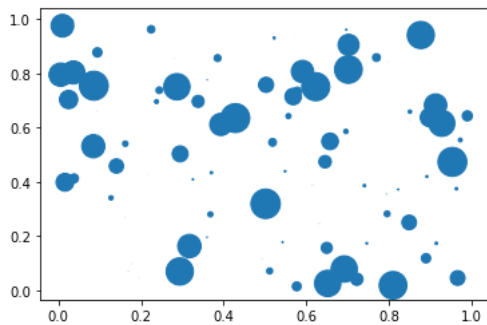
```
np.random.seed(42) # to make this notebook's output reproducible

x, y = np.random.rand(2, 100)
plt.scatter(x, y)
plt.show()
```



You may also optionally provide the scale of each point.

```
x, y, scale = np.random.rand(3, 100)
scale = 500 * scale ** 5
plt.scatter(x, y, s=scale)
plt.show()
```

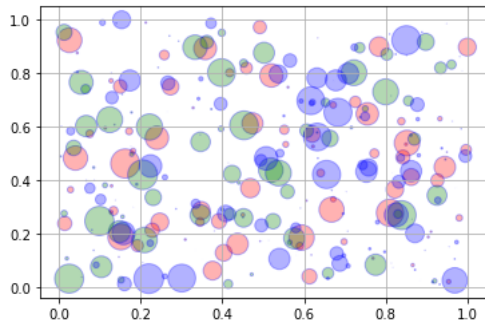


And as usual there are a number of other arguments you can provide, such as the fill and edge colors and the alpha level.

```
for color in ['red', 'green', 'blue']:
    n = 100
    x, y = np.random.rand(2, n)
    scale = 500.0 * np.random.rand(n) ** 5
    plt.scatter(x, y, s=scale, c=color, alpha=0.3, edgecolors='blue')

plt.grid(True)

plt.show()
```

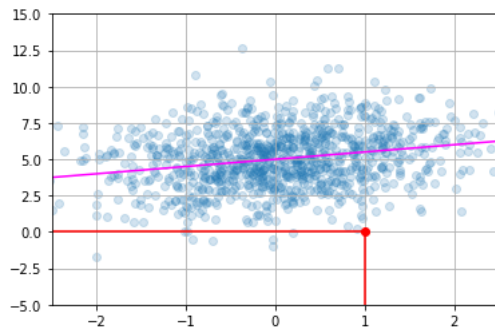


✓ Lines

You can draw lines simply using the `plot` function, as we have done so far. However, it is often convenient to create a utility function that plots a (seemingly) infinite line across the graph, given a slope and an intercept. You can also use the `hlines` and `vlines` functions that plot horizontal and vertical line segments. For example:

```
def plot_line(axis, slope, intercept, **kwargs):
    xmin, xmax = axis.get_xlim()
    plt.plot([xmin, xmax],
             [xmin*slope+intercept, xmax*slope+intercept],
             **kwargs)

x = np.random.randn(1000)
y = 0.5*x + 5 + np.random.randn(1000) * 2
plt.axis([-2.5, 2.5, -5, 15])
plt.scatter(x, y, alpha=0.2)
plt.plot(1, 0, "ro")
plt.vlines(1, -5, 0, color="red")
plt.hlines(0, -2.5, 1, color="red")
plot_line(axis=plt.gca(), slope=0.5, intercept=5, color="magenta")
plt.grid(True)
plt.show()
```

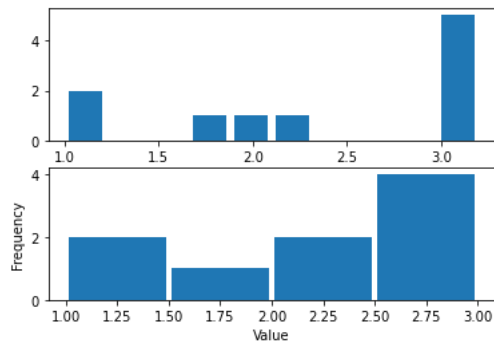


✓ Biểu đồ tần suất (Histograms)

```
data = [1, 1.1, 1.8, 2, 2.1, 3.2, 3, 3, 3, 3]
plt.subplot(211)
plt.hist(data, bins = 10, rwidth=0.8)

plt.subplot(212)
plt.hist(data, bins = [1, 1.5, 2, 2.5, 3], rwidth=0.95)
plt.xlabel("Value")
plt.ylabel("Frequency")

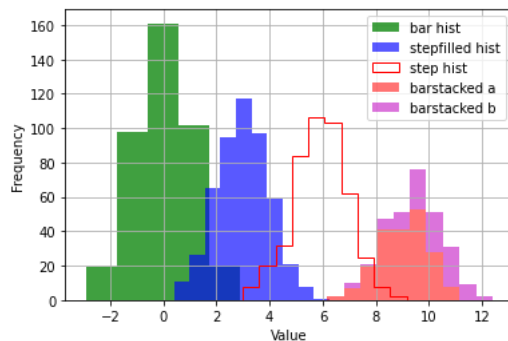
plt.show()
```

```
data1 = np.random.randn(400)
data2 = np.random.randn(500) + 3
data3 = np.random.randn(450) + 6
data4a = np.random.randn(200) + 9
data4b = np.random.randn(100) + 10

plt.hist(data1, bins=5, color='g', alpha=0.75, histtype='bar', # default type
         label='bar hist')
plt.hist(data2, color='b', alpha=0.65, histtype='stepfilled',
         label='stepfilled hist')
plt.hist(data3, color='r', histtype='step', label='step hist')
plt.hist((data4a, data4b), color=('r','m'), alpha=0.55, histtype='barstacked',
         label=('barstacked a', 'barstacked b'))

plt.xlabel("Value")
plt.ylabel("Frequency")
plt.legend()
plt.grid(True)
plt.show()
```



✧ Hình ảnh

Việc đọc, tạo và vẽ hình ảnh trong matplotlib khá đơn giản.

Để đọc một hình ảnh, chỉ cần import module `matplotlib.image`, và gọi hàm `imread` của nó, truyền vào tên tệp. Nó trả về dữ liệu hình ảnh dưới dạng một mảng NumPy. Hãy thử với hình ảnh `my_square_function.png` chúng ta đã lưu trước đó.

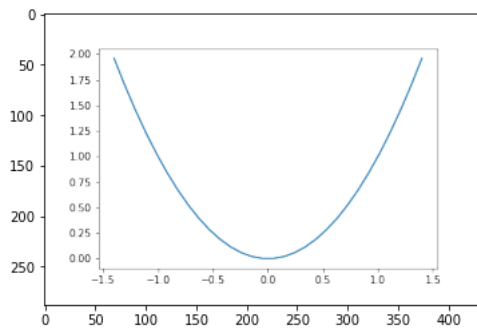
```
import matplotlib.image as mpimg

img = mpimg.imread("my_square_function.png")
print(img.shape, img.dtype)
```

```
(288, 432, 4) float32
```

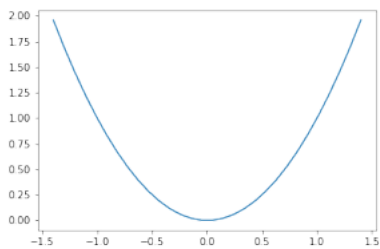
We have loaded a 288x432 image. Each pixel is represented by a 4-element array: red, green, blue, and alpha levels, stored as 32-bit floats between 0 and 1. Now all we need to do is to call `imshow`:

```
plt.imshow(img)
plt.show()
```



Tadaaa! You may want to hide the axes when you are displaying an image:

```
plt.imshow(img)
plt.axis('off')
plt.show()
```



Under the hood, `imread()` uses the Python Image Library (PIL), and Matplotlib's documentation now recommends using PIL directly:

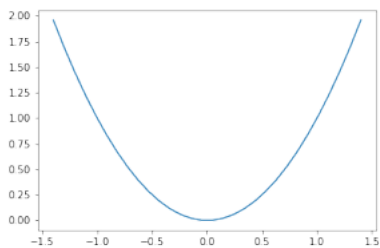
```
import PIL

img = np.asarray(PIL.Image.open("my_square_function.png"))
print(img.shape, img.dtype)

(288, 432, 4) uint8
```

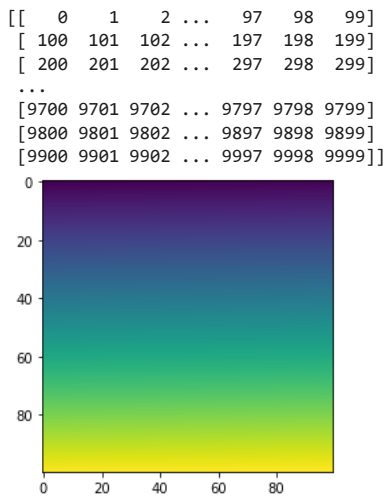
Note that the array now contains unsigned 8-bit integers (from 0 to 255), but that's fine as `plt.imshow()` supports this format as well:

```
plt.imshow(img)
plt.axis('off')
plt.show()
```



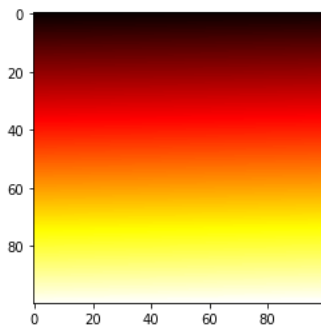
You can also generate an image from scratch:

```
img = np.arange(100*100).reshape(100, 100)
print(img)
plt.imshow(img)
plt.show()
```



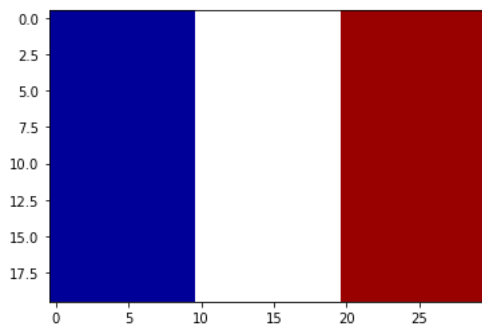
As we did not provide RGB levels, the `imshow` function automatically maps values to a color gradient. By default, the color gradient goes from blue (for low values) to yellow (for high values), but you can select another color map. For example:

```
plt.imshow(img, cmap="hot")
plt.show()
```



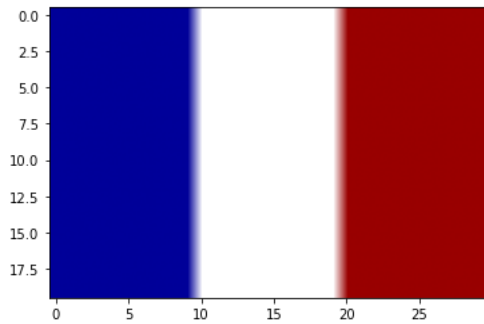
You can also generate an RGB image directly:

```
img = np.empty((20,30,3))
img[:, :10] = [0, 0, 0.6]
img[:, 10:20] = [1, 1, 1]
img[:, 20:] = [0.6, 0, 0]
plt.imshow(img)
plt.show()
```



Since the `img` array is just quite small (20x30), when the `imshow` function displays it, it grows the image to the figure's size. Imagine stretching the original image, leaving blanks between the original pixels. How does `imshow` fill the blanks? Well, by default, it just colors each blank pixel using the color of the nearest non-blank pixel. This technique can lead to pixelated images. If you prefer, you can use a different interpolation method, such as [bilinear interpolation](#) to fill the blank pixels. This leads to blurry edges, which may be nicer in some cases:

```
plt.imshow(img, interpolation="bilinear")
plt.show()
```



✓ Ảnh động (Animations)

Mặc dù matplotlib chủ yếu được sử dụng để tạo hình ảnh tĩnh, nó cũng có khả năng hiển thị các ảnh động. Đầu tiên, bạn cần import `matplotlib.animation`.

```
import matplotlib.animation as animation
```

In the following example, we start by creating data points, then we create an empty plot, we define the update function that will be called at every iteration of the animation, and finally we add an animation to the plot by creating a `FuncAnimation` instance.

The `FuncAnimation` constructor takes a figure, an update function and optional arguments. We specify that we want a 50-frame long animation, with 100ms between each frame. At each iteration, `FuncAnimation` calls our update function and passes it the frame number `num` (from 0 to 49 in our case) followed by the extra arguments that we specified with `fargs`.

Our update function simply sets the line data to be the first `num` data points (so the data gets drawn gradually), and just for fun we also add a small random number to each data point so that the line appears to wiggle.

```
x = np.linspace(-1, 1, 100)
y = np.sin(x**2*25)
data = np.array([x, y])

fig = plt.figure()
line, = plt.plot([], [], "r-") # start with an empty plot
plt.axis([-1.5, 1.5, -1.5, 1.5])
```